

IDS Exam Cheat Sheet

Precision / Recall / Accuracy / Cost / Hoeffding

1. Confusion Matrix

	Predicted = 1	Predicted = 0
Actual = 1	TP (correct, cost=80)	FN (missed fraud! cost=900)
Actual = 0	FP (false alarm, cost=150)	TN (correct, cost=0)

2. Precision / Recall / Accuracy

Metric	Formula	Question
Precision	$TP / (TP + FP)$	How accurate are alarms?
Recall	$TP / (TP + FN)$	How many frauds caught?
Accuracy	$(TP+TN) / \text{total}$	Overall correct rate?

Manual code (no sklearn):

```
y_pred = (y_proba >= threshold).astype(int)
TP = np.sum((y_pred==1) & (y_true==1))
FP = np.sum((y_pred==1) & (y_true==0))
FN = np.sum((y_pred==0) & (y_true==1))
TN = np.sum((y_pred==0) & (y_true==0))
```

sklearn version:

```
from sklearn.metrics import precision_score, recall_score, accuracy_score
precision_score(y_true, y_pred, pos_label=1)
recall_score(y_true, y_pred, pos_label=1)
accuracy_score(y_true, y_pred)
```

3. Cost Function (Part 1)

```
def problem2_avg_cost(y_true, y_predict_proba, threshold):
    y_pred = (y_predict_proba >= threshold).astype(int)
    TP = 0; FP = 0; FN = 0
    for i in range(len(y_true)):
        if y_true[i]==1 and y_pred[i]==1: TP += 1
        elif y_true[i]==1 and y_pred[i]==0: FN += 1
        elif y_true[i]==0 and y_pred[i]==1: FP += 1
    cost = (TP*80 + FP*150 + FN*900) / len(y_true)
    return cost
```

4. Threshold Optimization

Part 2 - Minimize Cost:

```
best_threshold = 0
min_cost = float('inf')
for t in np.arange(0, 1.01, 0.01):
    cost = problem2_avg_cost(y_true, y_proba, t)
    if cost < min_cost:
        min_cost = cost
        best_threshold = t
problem2_threshold = best_threshold
problem2_cost_val = min_cost
```

Part 3 - Maximize Accuracy:

```
max_acc = 0; best_t = 0 for t in np.arange(0, 1.01, 0.01): y_pred = (y_proba >=
t).astype(int) acc = np.sum(y_pred == y_true) / len(y_true) if acc > max_acc: max_acc
= acc; best_t = t problem3_threshold = best_t
```

5. Hoeffding Confidence Interval (Part 4)

Formula: $\text{mean} \pm (b-a) * \sqrt{\log(2/\delta) / (2n)}$

Where: $a=0$, $b=900$, $\delta=0.05$ (95% confidence), n =number of test samples

```
y_pred_test = (PROBLEM2_y_pred_proba_test >= problem2_threshold).astype(int) costs =
[] for i in range(len(PROBLEM2_y_true_test)): if PROBLEM2_y_true_test[i]==1 and
y_pred_test[i]==1: costs.append(80) elif PROBLEM2_y_true_test[i]==0 and
y_pred_test[i]==1: costs.append(150) elif PROBLEM2_y_true_test[i]==1 and
y_pred_test[i]==0: costs.append(900) else: costs.append(0) costs = np.array(costs) n
= len(costs) mean_cost = np.mean(costs) a, b, delta = 0, 900, 0.05 epsilon = (b-a) *
np.sqrt(np.log(2/delta) / (2*n)) problem4_ci_lower = mean_cost - epsilon
problem4_ci_upper = mean_cost + epsilon
```

Assumptions (write in markdown cell):

1. Test samples are i.i.d.
2. Each sample cost is bounded: cost_i in $[0, 900]$
3. Use test data, NOT validation data
4. Threshold from Part 2 (cost-optimal threshold)